

# convert\_fx#

[https://pytorch.org/docs/stable/generated/torch.quantization.quantize\\_fx.](https://pytorch.org/docs/stable/generated/torch.quantization.quantize_fx.html)

## Description:

Convert a calibrated or trained model to a quantized model

`graph_module (*)` – A prepared and calibrated/trained model (GraphModule)

`convert_custom_config (*)` – custom configurations for convert function. See `ConvertCustomConfig` for more details

`_remove_qconfig (*)` – Option to remove the qconfig attributes in the model after convert.

`qconfig_mapping (*)` –

## Function Signature

---

```
class
torch.ao.quantization.quantize_fx.convert_fx(graph_module,
convert_custom_config=None, _remove_qconfig=True,
qconfig_mapping=None, backend_config=None,
keep_original_weights=False)[source]#
```

Parameters

**backend\_config** (BackendConfig): A configuration for the backend which describes how

Returns

Return type

**Returns:**

GraphModule

## Code Examples

---

```
qconfig_mapping = QConfigMapping
    .set_global(qconfig_from_prepare)
    .set_object_type(torch.nn.functional.add, None) # skip
quantizing torch.nn.functional.add
    .set_object_type(torch.nn.functional.linear,
qconfig_from_prepare)
    .set_module_name("foo.bar", None) # skip quantizing module
"foo.bar"
```

```
# prepared_model: the model after prepare_fx/prepare_qat_fx and
calibration/training
# convert_fx converts a calibrated/trained model to a quantized
model for the
# target hardware, this includes converting the model first to a
reference
# quantized model, and then lower the reference quantized model
to a backend
# Currently, the supported backends are fbgemm (onednn), qnnpack
(xnnpack) and
# they share the same set of quantized operators, so we are
using the same
# lowering procedure
#
# backend_config defines the corresponding reference quantized
module for
# the weighted modules in the model, e.g. nn.Linear
# TODO: add backend_config after we split the backend_config for
fbgemm and qnnpack
# e.g. backend_config = get_default_backend_config("fbgemm")
quantized_model = convert_fx(prepared_model)
```

# Detailed Documentation

---

## Documentation

Convert a calibrated or trained model to a quantized model

`graph_module (*)` – A prepared and calibrated/trained model (`GraphModule`)

`convert_custom_config (*)` – custom configurations for convert function. See `ConvertCustomConfig` for more details

`_remove_qconfig (*)` – Option to remove the `qconfig` attributes in the model after convert.

`qconfig_mapping (*)` –

config for specifying how to convert a model for quantization.

The keys must include the ones in the `qconfig_mapping` passed to `prepare_fx` or `prepare_qat_fx`, with the same values or `None`. Additional keys can be specified with values set to `None`.

For each entry whose value is set to `None`, we skip quantizing that entry in the model:

operators should be quantized in the backend, this includes quantization mode support (static/dynamic/weight\_only), dtype support (quint8/qint8 etc.), observer placement for each operators and fused operators. See `BackendConfig` for more details

A quantized model (`torch.nn.Module`)

---

